

Las dos interfaces y cómo se implementan

Identificable

¿Qué representa?

Un contrato de identidad. Cualquier objeto que lo implemente puede ser identificado por un código numérico. Esto es lo que le permite al repositorio genérico buscar objetos sin importar de qué tipo son.

```
public interface Identificable {
    int getCodigo();
}
```

¿Quién la implementa?

Clase	¿Cómo?
Categoria	directamente: implements Identificable
Articulo	directamente: implements Identificable
ArticuloAlimenticio	por herencia de Articulo
ArticuloElectronico	por herencia de Articulo

ArticuloAlimenticio y ArticuloElectronico no necesitan escribir implements Identificable ni redefinir getCodigo() — lo heredan de Articulo automáticamente.

Calculable

¿Qué representa?

Un contrato de comportamiento. Todo artículo concreto (que tenga IVA propio) debe saber calcular su precio final. La interfaz **no define cómo**, solo exige que lo hagan.

```
public interface Calculable {
    double calcularPrecioFinal();
}
```

¿Quién la implementa y cómo la resuelve cada uno?

```
// ArticuloAlimenticio – IVA del 21%
public class ArticuloAlimenticio extends Articulo implements Calculable {
    private static final double IVA = 0.21;

    @Override
    public double calcularPrecioFinal() {
        return getPrecio() * (1 + IVA);
    }
}

// ArticuloElectronico – IVA del 10.5%
public class ArticuloElectronico extends Articulo implements Calculable {
    private static final double IVA = 0.105;

    @Override
    public double calcularPrecioFinal() {
        return getPrecio() * (1 + IVA);
    }
}
```

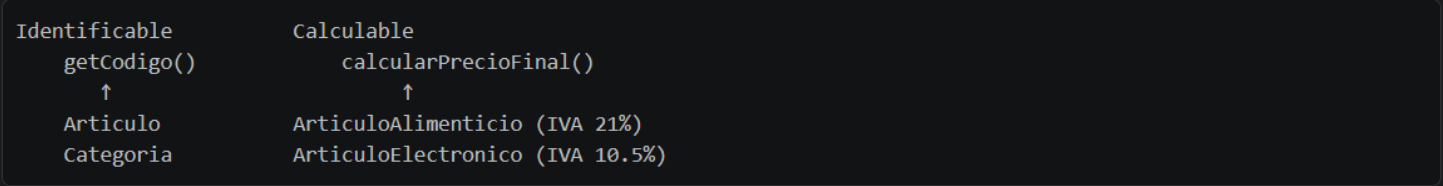
Mismo método, distinta lógica → eso es **polimorfismo de comportamiento**.

Diferencia clave entre ambas

	Identificable	Calculable
--	---------------	------------

Propósito	Identidad (¿quién sos?)	Comportamiento (¿qué hacés?)
La implementa	<code>Categoria</code> y <code>Articulo</code> (base)	Solo los subtipos concretos
Herencia	Los subtipos la heredan sin hacer nada	Cada subtipo la implementa distinto
Usa en	<code>Repositorio<T extends Identificable></code>	<code>toString()</code> y lógica de precios

Diagrama conceptual



`Articulo` no implementa `Calculable` porque la clase base no sabe qué IVA aplicar — esa decisión se delega a cada subtipo. Es un diseño intencional: forzar que cada tipo concreto defina su propia regla de precio.